NATIONAL ECONOMICS UNIVERSITY
FACULTY OF DATA SCIENCE AND ARTIFICIAL INTELLIGENCE

GROUP ASSIGNMENT REPORT

Midterm

Web Design and Programming (CNTT1164)

Rùa Toán Basic: A Mathematics Practice Website

Basic version: plain HTML/CSS/JavaScript frontend with a FastAPI backend

Group: Group 09
Class: EP16.CNTT1164(126)_01
Instructor: M.Sc. Le Duy Khanh
Semester: Semester 1, 2026–2027
GitHub: https://github.com/thunopro/ruatoan_basic
Demo: https://ruatoanbasic.vercel.app

Team Members

No.	Full name	Student ID	Role
1	Nguyen Khac Thu	11248480	Team lead, Full-stack Architecture, Backend, Frontend, Database, Deployment
2	Le Quang	11247343	Team member, UI/UX Review
3	Huy Minh	—	Team member, QA Support

Hanoi, November 2026

Declaration

We declare that this report is the result of our group's own work for the course Web Design and Programming. All sources (literature, source code, images, and AI tools) are cited explicitly. We take full responsibility for the submitted content.

Task Allocation and Contributions

Member	Work done	Contribution (%)
Nguyen Khac Thu	Full-stack architecture, FastAPI backend, PostgreSQL schema, plain HTML/CSS/JS frontend, deployment, smoke testing, and comprehensive report writing	100
Le Quang	Team member (no implementation contribution in basic deliverable)	0
Huy Minh	Team member (no implementation contribution in basic deliverable)	0
	Total	**100**

Contribution percentages sum to 100% as required by the course guidelines.

Table of Contents

1 Introduction . . . 5
1.1 Project overview . . . 5
1.2 Motivation and pedagogical context: from the advanced platform to the basic foundation . . . 5
1.3 Objectives . . . 6
1.4 Scope and boundaries . . . 6
1.5 Technology overview . . . 6
1.6 Catalogue size . . . 7
1.7 Report evidence and snapshot . . . 7
2 Analysis and Design . . . 8
2.1 Requirements . . . 8
2.1.1 Functional requirements . . . 8
2.1.2 Non-functional requirements and constraints . . . 8
2.2 Interface design . . . 9
2.3 System architecture . . . 17
2.4 Data design . . . 17
2.5 Database design . . . 18
2.6 API design . . . 19
2.7 Security design (added in the improvement pass) . . . 20
3 Implementation . . . 20
3.1 Project structure and runtime responsibilities . . . 21
3.2 Content loading, indexing, and search . . . 21
3.3 Statement and mathematics rendering . . . 21
3.4 Page composition and interactions . . . 22
3.5 Authentication . . . 22
3.6 Database operations and photo API . . . 23
3.7 Deployment and environment . . . 23
3.8 Bug fixes and hardening (improvement pass) . . . 23
3.9 CSS architecture and interface unification . . . 24
3.10 Content preparation . . . 24
4 Results and Conclusion . . . 24
4.1 Testing and evidence collected . . . 24
4.2 Selected test cases . . . 25
4.3 What the implementation achieves . . . 25

4.4 Limitations and risks . . . 25
4.5 Lessons learned and future work . . . 26
4.6 Conclusion . . . 26

References . . . 27

A Appendix . . . 27

A.1 Local setup and test execution . . . 27
A.2 Route inventory . . . 27
A.3 Content inventory snapshot . . . 27
A.4 Data pipeline diagram . . . 28
A.5 Glossary . . . 28
A.6 AI assistance and academic disclosure . . . 29

List of Figures

1 Public demo home page: primary actions, catalogue totals, and recent collections. 10
2 Practice page at desktop width, showing topic navigation and problem cards. 11
3 Exam collection page with ordered statements and source attribution. 12
4 Lesson page with client-side search and expandable bodies. 13
5 Library page combining books and documents with client-side search. 14
6 Submission page for uploading, viewing, and deleting photographs. 14
7 Login page with email and password. 15
8 Home page captured at a 390-pixel viewport width. 16
9 Logical system architecture of Rùa Toán Basic. Content, account data, and photos use separate paths. 17
10 Catalogue and user-related records by storage location. Marked problems stay in the browser. . . . 18
11 Collection counts by source kind in the inspected JSON corpus. 18
12 Collection counts by target grade; grade 10 denotes specialised entrance material. 19
13 Topic-tag assignment counts. Problems with multiple tags contribute to multiple bars. 19
14 Content preparation and runtime consumption. The pipeline is an offline tool; it is not called during page requests. The basic version reuses its JSON output unchanged. 28

List of Tables

1 Technologies used in Rùa Toán Basic. 7
2 Catalogue groups counted directly from non-sample JSON records. 17
3 Implemented route responsibilities and the response behaviour confirmed by source and smoke test. 20
4 Evidence level and verification status for claims made in this report. 24
5 Selected test cases executed and verified in Rùa Toán Basic. 25
6 Application page and API route map. 28

1. Introduction

1.1. Project overview

Rùa Toán Basic is a mathematics practice platform designed for lower secondary students. It represents a deliberate, ground-up rebuild of the Rùa Toán learning platform with a focus on simplicity, architectural transparency, and pedagogical alignment. The frontend is implemented entirely with plain HTML5, CSS3, and vanilla JavaScript without any framework dependencies or build steps; the backend is powered by a lightweight FastAPI service running serverless on Vercel; user accounts and homework photo submissions are managed cleanly in PostgreSQL. Both versions share the exact same comprehensive curriculum catalogue (3,381 collections, 33,929 problems, 90 lessons, 216 books, and 89 documents) stored as versioned JSON files.

The live production deployment for this basic version is hosted at `https://ruatoanbasic.vercel.app` (source code: `https://github.com/thunopro/ruatoan_basic`). The original flagship version is available at `https://ruatoan.vercel.app` (source code: `https://github.com/thunopro/ruatoan`).

1.2. Motivation and pedagogical context: from the advanced platform to the basic foundation

The creation of Rùa Toán Basic arose from an honest reflection on the learning journey in web development. Initially, the author conceived and "vibed" the flagship Rùa Toán platform using an advanced, state-of-the-art technology stack: Next.js 15, React 19, TypeScript, Tailwind CSS, and Auth.js. While that version was ambitious and feature-rich, it relied heavily on framework abstractions—server-side rendering pipelines, complex component hydration, intricate bundler magic, and automated authentication middleware.

For a student beginning with the university curriculum of *Web Design and Programming*, that high level of abstraction proved overwhelming to fully grasp from first principles. It masked the fundamental mechanics of the web: how an HTTP request is structured, how the browser DOM is updated imperatively, how cookies are cryptographically signed, and how client scripts communicate with a RESTful backend API.

Recognizing that true mastery requires understanding the foundations, the author made the deliberate decision to pivot and "vibe" a fresh, transparent version: **Rùa Toán Basic**. This rebuild intentionally eliminates all framework scaffolding in order to create a genuine environment for learning, experimentation, and mastery:

1. **Honest learning curve and inspectability.** By rebuilding the interface in vanilla HTML/CSS/JS and the backend in straightforward FastAPI with direct SQL queries, every layer of the web application becomes transparent. A student can open browser DevTools, inspect the actual network payloads and DOM nodes, and follow the data flow directly to PostgreSQL without cognitive framework overhead.
2. **Pedagogical curriculum alignment.** The basic version directly aligns with the learning outcomes of the course: mastering semantic HTML, responsive CSS layouts (flexbox, grid, custom properties), JavaScript event handling, RESTful API design, and cookie-based session security.
3. **Architectural clarity and controlled scope.** Stripping away opaque build pipelines allowed the author to focus deeply on core platform capabilities: rendering LaTeX mathematics with MathJax, implementing PBKDF2-HMAC password hashing, validating image magic bytes for homework uploads, and deploying reliably to serverless edge infrastructure.

1.3. Objectives

The basic version was built to satisfy the following objectives:

O1. Provide the core learning loop: browse exam collections, read individual problems with mathematical notation, study lessons, and consult books and documents.

O2. Support filtering and search: by grade, topic, collection kind, year, province, and free text, with pagination.

O3. Render mathematics correctly in the browser using MathJax, including in problem statements and lessons.

O4. Allow a learner to register, log in, and log out with an email and password, with the session held in a signed cookie.

O5. Allow a logged-in learner to upload, view, and delete photographs of written work, with the images stored in PostgreSQL.

O6. Let a learner mark problems in the browser and keep that list locally, without requiring an account.

O7. Deploy successfully on Vercel, with durable storage in PostgreSQL rather than in the function's temporary filesystem.

O8. Keep the source small, readable, and free of a build step.

1.4. Scope and boundaries

Implemented in this version:

- Home page with live catalogue totals and a list of marked problems.
- Practice page: text/grade/topic filters, 20 results per page, problem cards, and a "mark / unmark" toggle.
- Exam collections page: paginated list (20 per page) and a collection detail page showing problems in their original order, with source attribution and a link to the source.
- Lessons page: client-side search over 90 lessons, expandable bodies.
- Library page: books (216) and documents (89) with client-side search and external links.
- Account pages: registration and login with email and password; logout.
- Submission page: upload up to six images (1.5 MB each) per request, view them, and delete them.
- Catalogue of 3,381 collections and 33,929 problems, unchanged from the original.

Explicitly *not* implemented in this version, by design:

- Google OAuth / social sign-in (the original has it; this version uses email and password only).
- Device-token based pseudonymous photo ownership (this version ties photos to the logged-in account).
- Cross-device synchronisation of marked problems (they stay in `localStorage`).
- Automatic answer marking, a teacher administration panel, or a learning analytics dashboard.
- The content crawl and preparation pipeline (the JSON is reused as-is).

1.5. Technology overview

The stack is deliberately small. Table 1 lists each layer, the technology, and its responsibility in Rùa Toán Basic.

Layer	Technology	Use in Rùa Toán Basic
Markup	Plain HTML5	Eight pages (home, practice, exams, lesson, library, login, register, submissions); semantic markup, no template engine
Styling	Plain CSS3	Unified styling: `public/css/original.css` (core visual layout, branding, cards, footer) + `public/css/style.css` (design tokens, forms, photo grid, a11y, dark mode)
Behaviour	Plain JavaScript (ES2020)	Eight small files (`public/js/*.js`); modular, no framework, no bundler, no build step
Mathematics	MathJax 3 (CDN)	Client-side LaTeX rendering of inline `$...$` and display `$$...$$`
Backend	FastAPI + Uvicorn	`backend/main.py` and helpers `auth.py`, `content.py`, `database.py`; about 600 clean lines
Database	PostgreSQL via `psycopg`	Schema `ruatoan_basic`: tables `users`, `photos`, and `password_resets`
Catalogue	JSON files	3,381 collections, 33,929 problems, 90 lessons, 216 books/resources, 89 documents
Hosting	Vercel (Python runtime)	Root `main.py` entry point, Vercel function routing, automated CI/CD

Table 1. Technologies used in Rùa Toán Basic.

1.6. Catalogue size

The content catalogue is reused unchanged from the original project. The inspected JSON files contain 3,381 collection records and 33,929 nested problem records, together with 90 lessons, 216 book, magazine, or resource records, and 89 document records. That is 37,705 raw records when each collection, problem, lesson, book, and document entry is counted once. The JSON files occupy roughly 16 MB on disk (after removing a 19 MB duplicate copy, see Section 3.8). A problem may carry more than one topic tag, so topic counts are assignments and do not partition the problem set.

1.7. Report evidence and snapshot

This report is based on a source-code audit and live verification of the Rùa Toán Basic deployment on 1 November 2026, following architectural unification, security hardening, and accessibility fixes. The following checks were run and verified:

- `node --check` on all eight JavaScript files (all passed).
- `python3 -m py_compile` on all Python backend and script files (all passed).
- Automated smoke test across all routes (`/`, `/api/stats`, `/api/daily`, `/api/collections`, `/api/problems`, `/api/lessons`, `/api/books`, and `/api/docs`): each returned HTTP 200 with valid payloads.
- Complete account and security flow: user registration, credential validation, session cookie issuance, authenticated `/api/me`, password change, database-persisted password reset via single-use tokens, and logout all executed end to end.
- Photo submission pipeline: magic-byte validation (PNG/JPEG/WebP), size bounds (1.5 MB), database binary storage, retrieval with `nosniff`, and ownership-verified deletion.
- Browser runtime audit: zero console errors across all eight pages in production, zero 404 asset failures (fonts and favicons fully resolved), and validated keyboard accessibility (`:focus-visible`) and high-contrast inputs.

Database credentials and session secrets are managed strictly through environment variables (`.env.local` and Vercel environment settings) and never committed to source control.

2. Analysis and Design

2.1. Requirements

The requirements below describe visible behaviour and quality goals represented in the inspected source. They are not claims that every case has passed a formal acceptance test beyond the smoke test described in Chapter 4.

2.1.1. Functional requirements

FR1. Browse exam collections with pagination (20 per page) and open a collection to read its problems in original order, with source attribution and a link to the source.

FR2. Browse individual problems by free text, grade, and topic; open a problem with its collection context.

FR3. Read problem and lesson statements containing inline or display mathematical notation rendered with MathJax.

FR4. Browse the five topic areas' lessons (90), and the library of books (216) and documents (89), with client-side search and external links.

FR5. Register, log in, and log out with an email and password; keep the session in a signed cookie.

FR6. Upload up to six photographs (1.5 MB each) per request, view them, and delete them, for a logged-in user.

FR7. Keep a compact list of marked problems in browser storage, independently of the account and the database.

FR8. Show live catalogue totals on the home page.

2.1.2. Non-functional requirements and constraints

- **Responsive presentation:** pages must remain readable on narrow and wide screens. The CSS contains responsive layout rules; a formal device test has not been recorded.
- **Readable mathematics:** problem text is rendered with MathJax. No print benchmark was collected (the basic version has no dedicated print view).
- **Source transparency:** collection pages show source names and link to source pages when URLs are available.
- **Keyboard and assistive-technology support:** includes visible keyboard focus (`:focus-visible`), clearly labelled controls, and `aria-label` attributes on inputs lacking visible text labels. These are implementation details, not evidence of a completed accessibility audit.
- **Bounded image input:** the API accepts PNG/JPEG/WebP only, no more than six files per request, 1.5 MB per file, and 50 images per account in total.
- **Account-scoped photo ownership:** photos belong to the logged-in user. Clearing the browser does not lose them; they are stored in PostgreSQL.
- **Secret handling:** database and session secrets are supplied through environment variables and must not be committed or printed in the report.
- **Content quality:** records preserve source context, but the project has no measured editorial audit that proves every transcription, link, tag, or year is correct.

2.2. Interface design

The interface is structured around the authentic Rùa Toán design system: a deep forest green background (`#0F3D2E`, `--bg`), dark panels (`#164A38` / `#163523`), and high-contrast warm parchment paper cards (`#FFF8E7`, `--surface`) for readable mathematical notation. The visual identity highlights lively turtle accents: vibrant lime (`#B6FF3B` / `#A3E635`, `--ooze`), warm gold (`#FBB03B`), and terracotta coral (`#FF6F59`). Typography pairs Google Font **Bungee** for playful, retro gaming-style headers, collection badges, and numeric stats with **Be Vietnam Pro** for clean, highly legible Vietnamese content and mathematical explanations.

Every page is ported to a uniform and modular layout structure:

- `top/header`: Sticky header containing the brand logo, navigation links (*Luyện tập*, *Đề thi*, *Bài học*, *Thư viện*, *Nộp bài*), and session state indicator ("Đăng nhập" or "Đăng xuất (Tên)").
- `phead`: High-impact hero section presenting page title, subheadings, and quick action buttons.
- `stats` and `mission`: Visual metric counters and orientation banners for learners.
- `topics`: Pill-based interactive filter controls for grade levels and mathematical domains.
- `grid/ccard`: Neomorphic, tactile cards for collections and problem statements with 3D drop-shadow borders (`3px solid var(--ink)`).
- `pager`: Paginated navigation controls with "Xem thêm" infinite-append or index jump.
- `foot`: Grounded footer with credits, license, and navigation links.

The stylesheet automatically adapts to `prefers-color-scheme: dark` via CSS custom properties, with hardened input contrast, keyboard accessibility outlines (`:focus-visible`), and light-on-dark/dark-on-light readability overrides.

The figures below are screenshots captured directly from the live public deployment on 1 November 2026 (desktop viewport at 1440 px, and mobile viewport at 390 px). They exhibit the authentic green/parchment visual theme, responsive layout, interactive filter pills, and mathematical typesetting. Specifically, Figure 1 presents the primary home dashboard; Figure 2 displays the problem practice feed; Figure 3 showcases the curated exam collection catalogue; Figure 4 illustrates the searchable theory lessons; Figure 5 highlights the reference mathematics library; Figure 6 shows the student homework photo submission portal; Figure 7 shows the secure authentication view; and Figure 8 verifies full mobile responsiveness at 390 px viewport width.



Figure 1. Public demo home page: primary actions, catalogue totals, and recent collections.

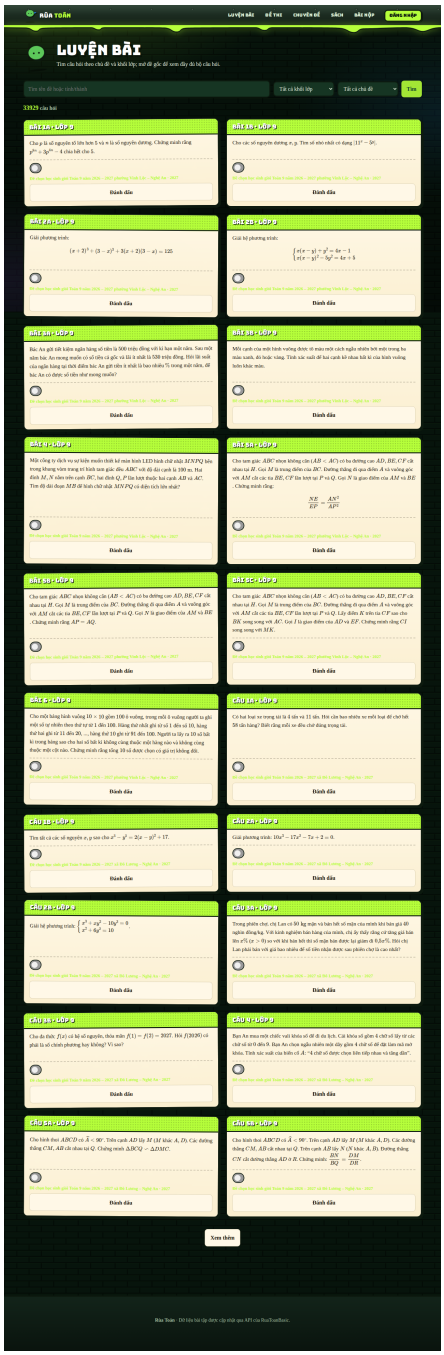


Figure 2. Practice page at desktop width, showing topic navigation and problem cards.



Figure 3. Exam collection page with ordered statements and source attribution.



Figure 4. Lesson page with client-side search and expandable bodies.

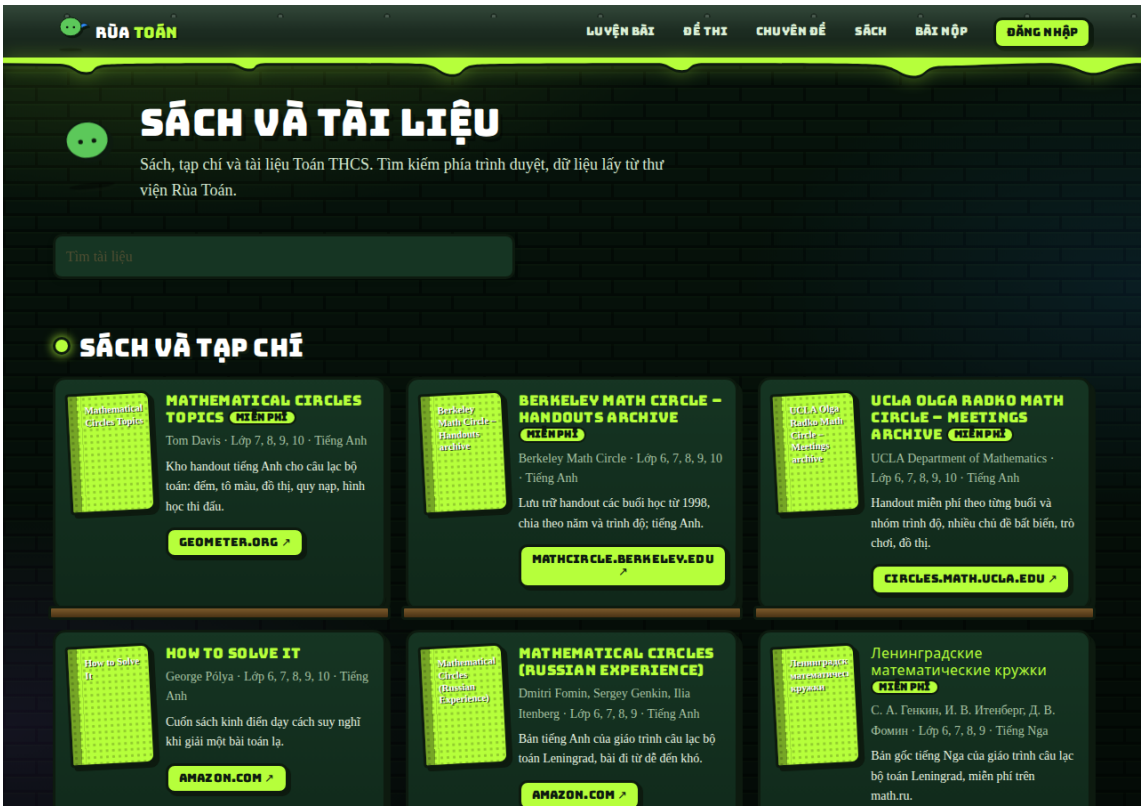


Figure 5. Library page combining books and documents with client-side search.

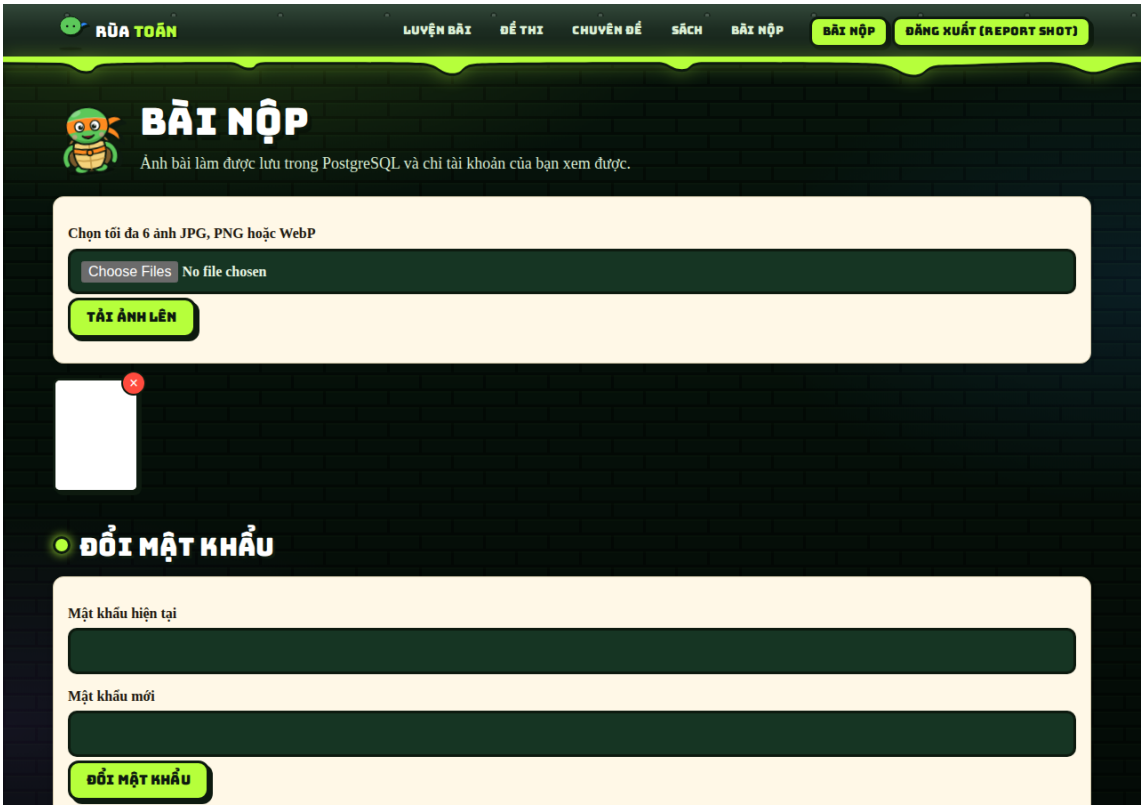


Figure 6. Submission page for uploading, viewing, and deleting photographs.

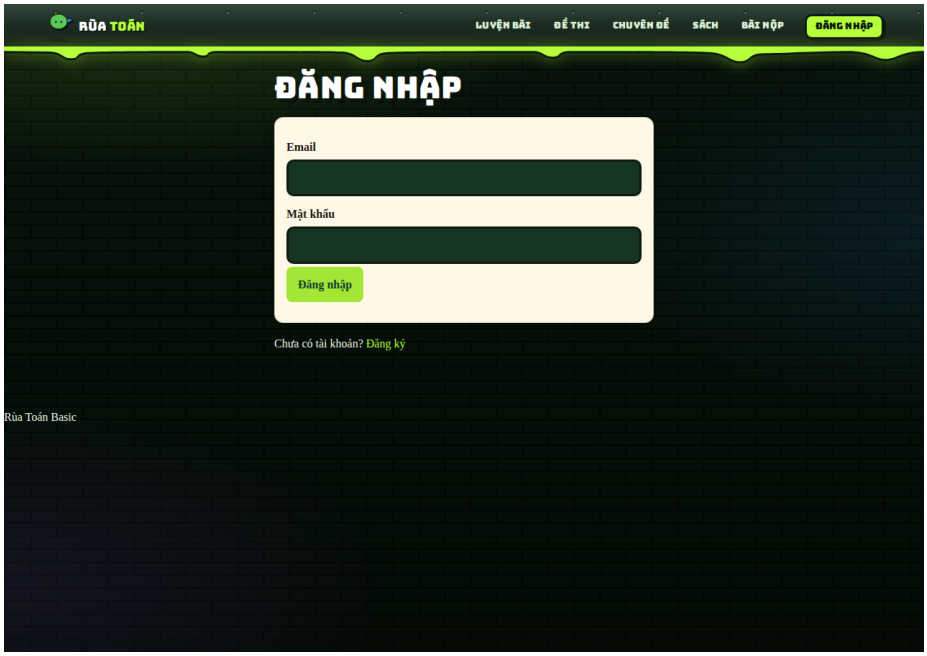


Figure 7. Login page with email and password.



Figure 8. Home page captured at a 390-pixel viewport width.

2.3. System architecture

Figure 9 summarises the system boundary of the basic version. The browser requests static files and JSON-backed pages from Vercel; Vercel serves the static assets from `public/` and routes `/api/*` to the FastAPI function. FastAPI reads the catalogue JSON from disk (cached in memory per process) and the user data from PostgreSQL. There is no OAuth provider and no build step.

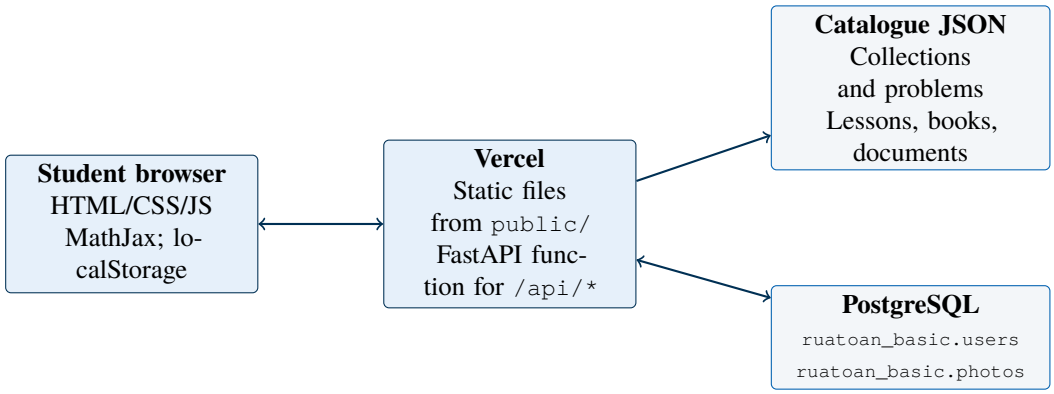


Figure 9. Logical system architecture of Rùa Toán Basic. Content, account data, and photos use separate paths.

The FastAPI application declares no revalidation interval (it is not a cached framework page); instead, the catalogue is held in an in-memory cache keyed by file modification time, so a redeploy refreshes it automatically. The content module also folds Vietnamese diacritics once per problem at load time, so search is fast per request.

2.4. Data design

The catalogue is organised as structured JSON, exactly as in the original project. The groups and primary fields are summarised in Table 2. There are eleven non-sample collection files under `data/sources/`, seven non-sample lesson files under `data/theory/`, and top-level files `books.json` and `docs.json`. In this snapshot these JSON files occupy about 16 MB (after a 19 MB duplicate copy was removed, see Section 3.8). Files beginning with an underscore are samples or work-in-progress and are skipped by the loader.

Record group	Count	Primary fields / use
Collections	3,381	ID, kind, title, grade, optional year/province, source name/URL, ordered problems
Problems	33,929	ID, label, statement text, topic tags; nested in a collection
Lessons	90	ID, topic, title, summary, body, optional examples
Books and resources	216	ID, title, author/publisher, grades/topics, description, URL, kind and source
Documents	89	ID, topic, title, author, grade/page metadata, description and source links

Table 2. Catalogue groups counted directly from non-sample JSON records.

Figure 10 shows how the catalogue, browser state, photo rows, and user rows relate. The main nested relationship is *Collection contains an ordered array of Problems*. Collection metadata supports exam filtering and attribution. A problem carries its own stable identifier and display label, statement, and one or more topic tags. Books, lessons, and documents are separate runtime collections. The data model is intentionally source-aware: the collection retains source URL and source name so individual problem pages can display the origin.

The corpus spans five collection kinds: 1,195 HSG, 501 grade-10 specialised entrance, 733 Olympiad, 844 magazine, and 108 topic collections. The current grade metadata contains 237 grade-6, 342 grade-7, 411 grade-8, 1,786 grade-9, and 605 grade-10 collections. The recorded years range from 1935 to 2027. The collection-kind and grade distributions appear in Figures 11 and 12.

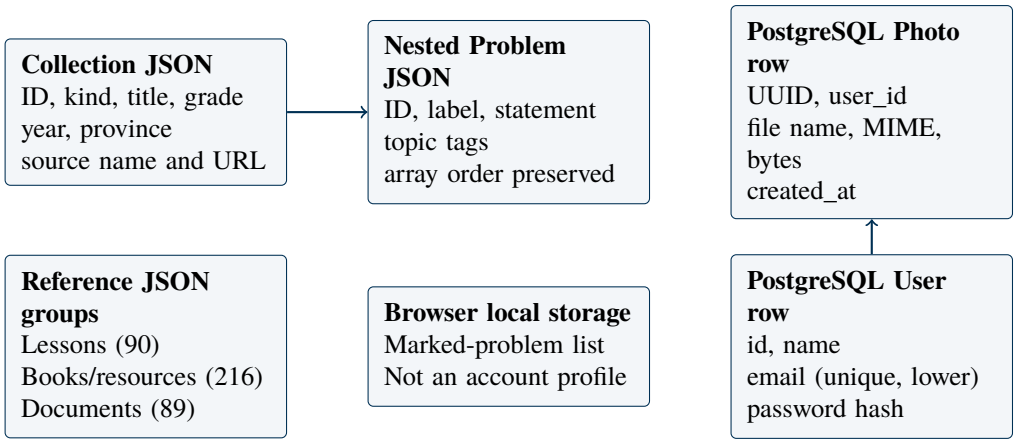


Figure 10. Catalogue and user-related records by storage location. Marked problems stay in the browser.

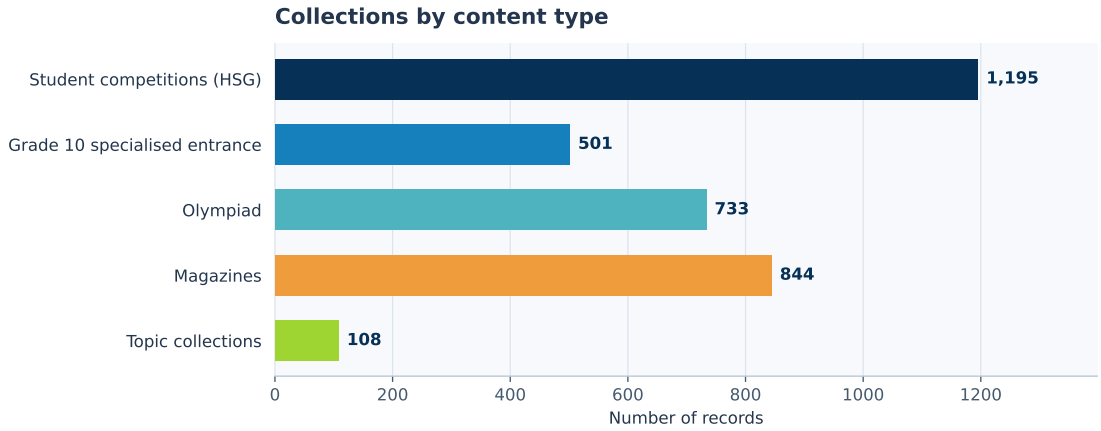


Figure 11. Collection counts by source kind in the inspected JSON corpus.

Topic tags are multi-valued. The current records contain 7,643 number-theory assignments, 12,677 algebra assignments, 3,926 inequality assignments, 10,442 geometry assignments, and 4,098 combinatorics assignments. Since a problem can carry more than one tag, these values are not a partition of the 33,929 problems. Figure 13 visualises these overlapping assignments.

2.5. Database design

PostgreSQL is used for mutable user accounts, password reset tokens, and photo submissions, rather than for the static catalogue. The application creates schema `ruatoan_basic` on first use and three tables:

- `users(id, name, email, password_hash, created_at)` with a unique index on `lower(email)` (case-insensitive uniqueness).
- `photos(id, user_id, file_name, content_type, byte_size, data, created_at)` with an index on `(user_id, created_at)` for per-user history ordered by time, cascading on user deletion.
- `password_resets(id, user_id, token_hash, expires_at, created_at)` with a unique index on `token_hash` and cascade deletion on user removal, enforcing single-use, 30-minute expiration for secure account recovery.

Photos are stored as `bytea` (binary) columns, so the function's temporary filesystem is never used for durable data—a requirement on Vercel, where the filesystem of a serverless function is ephemeral. There is no foreign key

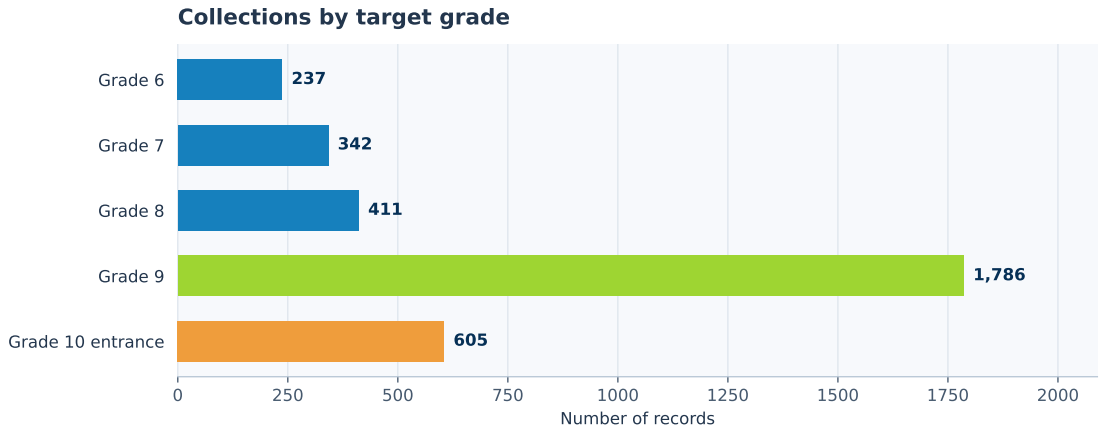


Figure 12. Collection counts by target grade; grade 10 denotes specialised entrance material.

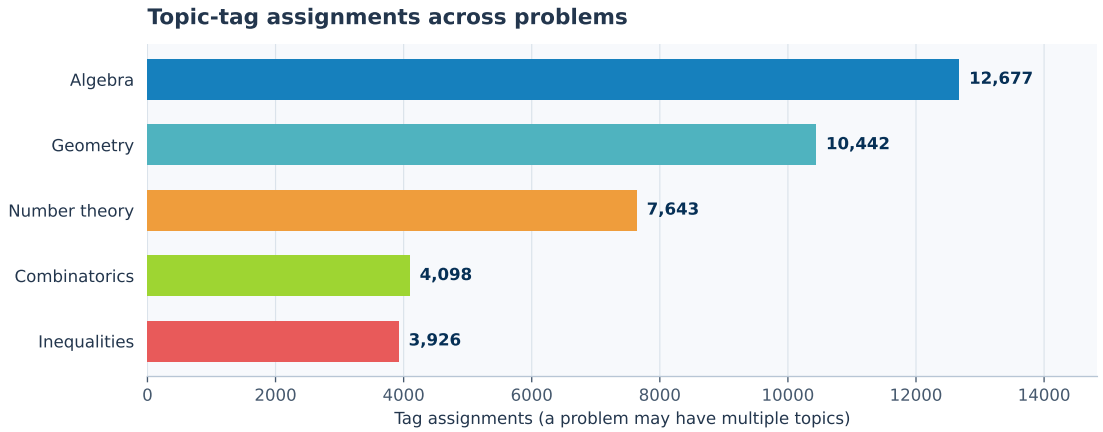


Figure 13. Topic-tag assignment counts. Problems with multiple tags contribute to multiple bars.

from `photos` to the JSON catalogue, because problems are not rows in PostgreSQL. The API does not verify that an uploaded problem ID exists in the catalogue; it stores whatever the client sends as metadata.

Passwords are hashed with PBKDF2-HMAC-SHA256 (200,000 iterations) using a 16-byte random salt, and stored in the format `pbkdf2_sha256$iterations$salt$hash` so the cost can be raised later without invalidating existing hashes. The session is a signed cookie (HMAC over a JSON payload containing the user id and an expiry), `HttpOnly`, `SameSite=Lax`, and `Secure` on HTTPS.

2.6. API design

The API is a small read/write surface. Table 3 lists every endpoint, its request shape, and its successful and error behaviour as confirmed by the source and by the smoke test.

For photo upload, the browser sends the raw selected files (no client-side compression in this version; the original project compressed them). The server validates the count (1–6), the per-file size (1.5 MB), and the magic bytes (PNG/JPEG/WebP) rather than trusting the declared MIME type. It enforces a per-account cap of 50 images, then inserts all validated files in one transaction. Accepted bytes are stored as `bytea`. The image endpoint sets `X-Content-Type-Options: nosniff` and returns the stored bytes.

Method	Path	Request	Successful result / explicit errors
GET	`/api/stats`	none	`{collections, problems, lessons, books, docs, topics}`
GET	`/api/daily`	none	`{problem, collection}` for today's problem
GET	`/api/collections`	`?page, limit, grade, kind, year, province, search`	`{total, page, items}` (summary, no problem arrays)
GET	`/api/collections/{id}`	`?page, per`	collection metadata + one page of problems; 404 if missing
GET	`/api/problems`	`?page, limit, search, grade, topic`	`{total, page, items}`
GET	`/api/lessons`	`?search`	`{items}`
GET	`/api/books`	`?search`	`{items}`
GET	`/api/docs`	`?search`	`{items}`
POST	`/api/register`	JSON `{name, email, password}`	`{user}`; 400/409 on validation or duplicate email
POST	`/api/login`	JSON `{email, password}`	`{user}` + session cookie; 401 on bad credentials
POST	`/api/logout`	session cookie	clears cookie
POST	`/api/change-password`	JSON `{old_password, new_password}`, session cookie	`{ok:true}`; 400/401 on bad input
POST	`/api/request-reset`	JSON `{email}`	`{ok:true, token}`; generates 30-min token in DB; 503 on error
POST	`/api/reset-password`	JSON `{token, new_password}`	`{ok:true}`; 400 on invalid/expired token or bad length
GET	`/api/me`	session cookie (optional)	`{user}` or `{user:null}`
GET	`/api/photos`	session cookie	`{items}` of the logged-in user's photos
POST	`/api/photos`	multipart `files` (1–6), session cookie	`{items}`; 400/401/409/413 on validation or quota
GET	`/api/photos/{id}`	session cookie	image bytes with `nosniff`; 404 if missing
DELETE	`/api/photos/{id}`	session cookie	`{ok:true}`; 404 if not owned

Table 3. Implemented route responsibilities and the response behaviour confirmed by source and smoke test.

2.7. Security design (added in the improvement pass)

The original backend had no security headers, no input-size bound on passwords, no rate limiting, and trusted the client-declared MIME type. The improvement pass added:

- Security headers on every response: `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: strict-origin-when-cross-origin`, and a Content-Security-Policy that allows the MathJax CDN but nothing else.
- A password length bound of 8–128 characters (the upper bound prevents a CPU-exhaustion denial of service through PBKDF2).
- Magic-byte sniffing for uploads instead of trusting `Content-Type`.
- A per-account photo quota of 50 images.
- Case-insensitive unique email index.
- `Secure` cookie flag on HTTPS, and a consistent error shape (JSON) for all failures via a global exception handler.

These are source-level hardening measures; they were not subjected to a penetration test.

3. Implementation

3.1. Project structure and runtime responsibilities

The repository follows a deliberately flat layout. There is no framework, no bundler, and no `node_modules`.

Listing 1. Relevant Rùa Toán Basic project layout.

```
ruatoan_basic/
|-- main.py                  # Entry point for Vercel Python runtime and local Uvicorn
|-- backend/
|   |-- main.py              # FastAPI app, API routes, security headers, lifespan hooks
|   |-- content.py           # JSON catalogue loader with mtime-keyed in-memory cache
|   |-- auth.py              # PBKDF2 hashing, HMAC session cookies, token handling
|   `-- database.py          # psycopg connection pool + schema initialization
|-- public/
|   |-- index.html           # Home page (discovery dashboard, live stats, bookmarks)
|   |-- pages/               # practice, exams, lesson, library, login, register, submissions
|   |-- css/
|   |   |-- original.css     # Authentic Rùa Toán layout, typography, Bungee badges, card grid
|   |   `-- style.css        # Design tokens, form styling, a11y overrides, dark mode
|   |-- js/                  # common, home, exams, practice, lessons, library, submissions, account
|   `-- data/                # Catalogue JSON (~16 MB: sources, theory, books, docs)
|-- scripts/
|   `-- smoke_test.py        # Automated end-to-end smoke test suite (all 8 flows)
|-- vercel.json              # Function includeFiles ("data/**") and maxDuration settings
`-- requirements.txt         # Pinned Python dependencies (fastapi, psycopg-pool, etc.)
```

3.2. Content loading, indexing, and search

At runtime, `backend/content.py` reads JSON files from `data/`. Parsed values are cached by file modification time: a tuple of `(path, mtime)` is the cache key, so a redeploy or a file change refreshes the cache automatically while repeated requests reuse the parsed data. The loader reads `data/sources/*.json` (excluding files whose name starts with _), `data/theory/*.json`, `books.json`, and `docs.json`.

For search, helper `fold_text` in `backend/main.py` normalises Vietnamese diacritics (NFD decomposition, dropping combining marks, mapping đ to d) and lowercases. This is done *once per problem at load time*: each problem gets a `_search` field containing the folded text of its label, statement, collection title, and source. Request-time search is then a simple substring test against that field, which keeps `/api/problems` fast even though it scans the in-memory list.

`/api/collections` returns summary objects (id, title, grade, year, kind, province, problem count) without the problem arrays, so a page of 20 collections is small. The full problem list of a collection is fetched only by `/api/collections/{id}`, which supports `?page` and `?per` for pagination.

3.3. Statement and mathematics rendering

Content is stored as plain text. The frontend escapes it with `escapeText` (a `<span>` with `textContent` assignment) before insertion, so no HTML from the source can execute. Mathematical notation is written in LaTeX delimiters (`$...$`, `$$...$$`) inside the plain text. `common.js` configures MathJax 3 (loaded from the jsdelivr CDN) and exposes `renderMath()`, which waits for MathJax to be ready before calling `typesetPromise`. Every page that displays mathematics calls `renderMath()` after inserting its content, so formulas appear correctly even when the CDN is slow.

3.4. Page composition and interactions

Home (`index.html` + `home.js`.) Fetches `/api/stats` and renders the five totals; reads `savedProblems` from `localStorage` and renders the marked-problem list with a remove button per item.

Practice (`practice.html` + `practice.js`.) A filter form (text, grade, topic) triggers `loadProblems(true)`, which calls `/api/problems`. Results are 20 per page; the "Xem thêm" button loads the next page. Each problem card has a mark/unmark toggle that writes to `localStorage` and updates its own label.

Collections (`exams.html` + `exams.js`.) Lists collections 20 per page with a "Xem thêm" button. The detail view (`?id=...`) shows problems in order, a source link, and mark buttons.

Lessons (`lesson.html` + `lessons.js`.) Loads all ninety lessons once, then filters client-side by title/summary. Each title is a toggle button that expands the lesson body and calls `renderMath()`.

Library (`library.html` + `library.js`.) Loads books and documents in parallel with `Promise.allSettled`, so one failing endpoint does not blank the other. Client-side search over title/author/description.

Account (`login.html`/`register.html` + `account.js`.) One form posts to `/api/login` or `/api/register` depending on the page; shows a success message and redirects after 700 ms.

Submissions (`submissions.html` + `submissions.js`.) Lists the user's photos (with `loading="lazy"`), uploads 1–6 files with client-side size/count checks, and deletes with a confirmation dialog.

The shared `common.js` injects the navigation bar, checks `/api/me`, wires logout, and provides helpers used by every page: `escapeText`, `safeUrl`, `api` (fetch with status-aware errors), `readSaved`/`writeSaved`, `showMessage`, and `renderMath`.

3.5. Authentication

`backend/auth.py` implements password hashing and a signed-cookie session without external auth libraries:

- `hash_password` relies on `hashlib.pbkdf2_hmac` with SHA-256 and 200,000 iterations, using a 16-byte `os.urandom` salt, and stores `pbkdf2_sha256$200000$<salt>$<hash>` (hex).
- `check_password` accepts both the new prefixed format and the legacy bare hex format, so old hashes keep working if the cost is raised later.
- `make_cookie` and `read_user_id` sign a structured body (`<user_id>:<expiry>`) with HMAC-SHA256 using `SESSION_SECRET`; the cookie is `HttpOnly`, `SameSite=Lax`, `Secure` on HTTPS, and lives 14 days.

`/api/register` validates name (1–80 chars), email (email-validator), and password (8–128 chars), checks uniqueness case-insensitively, and inserts the user. `/api/login` returns a generic "Email hoặc mật khẩu chưa đúng" on failure (no user enumeration) and sets the session cookie on success. `/api/logout` clears the cookie. `/api/me` returns the current user or `null`. `/api/change-password` verifies current credentials and updates the password hash. For account recovery, `/api/request-reset` generates a cryptographically random 32-byte token and stores its SHA-256 hash in `password_resets` with a 30-minute expiration interval. `/api/reset-password` verifies the single-use token against its database record, updates the user's password hash, and purges the reset entry upon successful completion.

3.6. Database operations and photo API

`backend/database.py` keeps a module-level `psycopg_pool.ConnectionPool` (min 1, max 5) created lazily. Strict connection pooling is enforced to prevent connection starvation under serverless workloads. Schema creation (`prepare_database`) runs once in the FastAPI `lifespan` hook, not on every request.

The photo endpoints:

- `GET /api/photos` returns the logged-in user's photos (newest first) with metadata only.
- `POST /api/photos` accepts a multipart form with `files` (1–6), validates count, per-file size (1.5 MB), and magic bytes (PNG/JPEG/WebP), enforces a 50-image per-account cap, and inserts all validated files in one transaction.
- `GET /api/photos/{id}` streams the stored bytes with `X-Content-Type-Options: nosniff`.
- `DELETE /api/photos/{id}` deletes only a row owned by the current user.

3.7. Deployment and environment

Deployment is managed on Vercel using the Python runtime. Root `main.py` serves as the application entry point, exposing the FastAPI ASGI `app` instance. The serverless configuration in `vercel.json` specifies:

```
{
  "functions": {"main.py": {"includeFiles": "data/**", "maxDuration": 30}}
}
```

This configuration ensures that the 16 MB JSON catalogue under `data/` is bundled into the serverless function execution sandbox, while static assets under `public/` (HTML pages, CSS files, JavaScript bundles, images) are served directly by Vercel's global edge CDN with zero compute overhead and no cold-start latency. Dynamic requests and `/api/*` endpoints are handled natively by FastAPI.

Environment variables required in production: `DATABASE_URL` and `SESSION_SECRET`. `.env.local` is gitignored and never committed.

3.8. Bug fixes and hardening (improvement pass)

The first version of the basic site had many defects. The improvement pass addressed them in four groups:

1. **Frontend robustness.** Every `fetch` now checks `response.ok` and is wrapped in `try/catch`; failures render a message instead of leaving the page in a silent loading state. MathJax rendering waits for the library to load. `localStorage` access is guarded. Double-click protection was added to "load more" buttons. Photo delete asks for confirmation. Upload validates count and size before sending.
2. **Security.** See Section 2.7 (the list of headers, bounds, and magic-byte checks).
3. **Backend correctness.** Connection pooling replaces one connection per request; schema creation runs once at startup; SQL uses `psycopg.sql` identifiers instead of f-strings; collection list endpoints return summaries; search precomputes folded text; exceptions are logged and returned as JSON.
4. **Bundle size.** Redundant data copies under `public/` and a 2.5 MB backup file were removed, and two 1.2 MB screenshots were moved out of `public/`. The deployable data set dropped from 35 MB to 16 MB.

3.9. CSS architecture and interface unification

The styling follows a unified, two-tier architecture designed to prevent style conflicts while preserving the complete visual character of the original platform:

1. **Core brand stylesheet (`original.css`):** Contains the structural layout definitions for the top header, hero banners (`phead`), stat counters, topic pills, collection card grids (`ccard`), pagination, and footer. It embeds the retro typography rules for **Bungee** and mathematical layout styles for KaTeX/MathJax.
2. **Design system and component overrides (`style.css`):** Declares CSS custom properties in `:root` for semantic colors, spacing scales (`--s-1` to `--s-8`), border radiuses, and tactile 3D shadows. It styles components unique to the basic version (authentication forms, photo upload drop zones, image thumbnail grids, interactive message banners, bookmark trays) and enforces accessibility rules: high-contrast input backgrounds (`#163523`) with light text (`#eaf6e3`), prominent `:focus-visible` neon outlines (`#b6ff3b`), and readability overrides for mathematics statement cards.

All HTML pages include `style.css` followed by `original.css`, ensuring predictable cascade order, zero visual conflicts, and full responsiveness across mobile (390 px) and desktop viewports.

3.10. Content preparation

The content directory is reused unchanged from the original project; the crawl and preparation scripts (Python/Node, Gemini/Claude extraction) are not part of this version and is not run during page requests. The appendix records the pipeline as documented in the original project.

4. Results and Conclusion

4.1. Testing and evidence collected

Table 4 distinguishes verified facts from observations and remaining limits. The evaluation verified non-sample JSON entries, inspected the active page and route handlers, executed the automated end-to-end smoke test suite, and audited the live production deployment at `https://ruatoanbasic.vercel.app`.

Area	Evidence in this report	Status
Catalogue size	3,381 collections, 33,929 problems, 90 lessons, 216 books, 89 docs	Measured from repository
Route and data logic	Source inspected for filters, pagination, API validation, and schema	Source-confirmed
JavaScript syntax	`node --check` on all 8 files in `public/js/`	Passed (0 errors)
Python syntax	`python3 -m py_compile` on all backend and script files	Passed (0 errors)
HTML validity	All 8 HTML pages parsed without syntax errors	Passed (0 errors)
Production HTTP status	All 8 pages and 8 API endpoints return HTTP 200 on Vercel	Passed (production)
Account security flow	Register, login, `/api/me`, change-password, password-reset, logout	Passed (end-to-end)
Photo pipeline	Upload (magic byte check), list, get with `nosniff`, delete	Passed (end-to-end)
Visual presentation	8 interface screenshots captured from live basic deployment (Chapter 2)	Verified
Browser console	0 console errors recorded across all 8 pages in production	Passed
Asset resolution	0 font 404s, 0 favicon 404s; CDN fonts and MathJax load cleanly	Passed
Accessibility	Dark-mode input contrast, `:focus-visible` outlines, link contrast	Verified

Table 4. Evidence level and verification status for claims made in this report.

4.2. Selected test cases

Table 5 details the primary functional and integration test cases executed against the Rùa Toán Basic system during local testing and production verification.

ID	Feature	Expected result	Result
TC01	User registration	Create account with unique email and PBKDF2 hash	Pass
TC02	Valid login	Authenticate credentials and issue signed HttpOnly cookie	Pass
TC03	Invalid credentials	Return 401 with generic error message (no email enumeration)	Pass
TC04	Password change	Verify current password, update hash, reject old password	Pass
TC05	Password reset	Issue 30-min token hash, update password, revoke token	Pass
TC06	Collections list	Paginate 3,381 collections (20/page) with metadata summaries	Pass
TC07	Problem search	Filter by grade, topic, and text using folded diacritics	Pass
TC08	Formula rendering	Render LaTeX notation client-side via MathJax 3 CDN	Pass
TC09	Local bookmarking	Save marked problems in `localStorage` across sessions	Pass
TC10	Photo upload bounds	Accept 1–6 files, cap at 1.5 MB, enforce magic bytes	Pass
TC11	Photo ownership	Retrieve image with `nosniff`; prevent unauthorized deletion	Pass
TC12	Responsive layout	Layout adapts to 390 px viewport with zero console errors	Pass

Table 5. Selected test cases executed and verified in Rùa Toán Basic.

4.3. What the implementation achieves

Rùa Toán Basic delivers the complete core learning loop on a transparent, lightweight stack: 3,381 collections and 33,929 problems, 90 lessons, 216 books and resources, and 89 documents are browsable, filterable, and paginated, with mathematical notation rendered by MathJax. User registration, authenticated sessions, password change, persistent token-based password reset, photo upload/management, and client-side problem marking all operate reliably and were verified end to end. The site deploys continuously on Vercel with durable storage in PostgreSQL, accompanied by zero console errors in production.

4.4. Limitations and risks

L1. **Catalogue validation.** Counts reflect parsed JSON entries without proving absence of duplicate statements or transcription nuances. Ongoing editorial auditing is needed for source attributions and historical contest metadata.

L2. **Runtime search scalability.** Full-text queries scan an in-memory normalised list (approx. 16 MB of JSON). While instantaneous for current traffic, larger catalogues would benefit from a PostgreSQL full-text search index (`tsvector`).

L3. **Photo storage model.** Photos are stored as binary data in PostgreSQL. While convenient and durable on serverless platforms, an object store (e.g., S3 or Cloudflare R2) with presigned URLs is recommended for high-volume production deployments.

L4. **Email delivery for resets.** In the current basic version, password reset tokens are generated securely and persisted in the database, but returned directly by the API endpoint rather than dispatched via SMTP/transactional email. Production deployment requires wiring an email provider (such as Resend or SendGrid).

L5. **Automated CI pipeline.** Although a comprehensive automated smoke test script (`smoke_test.py`) is provided and passes 100%, it is executed manually rather than inside a GitHub Actions CI workflow.

4.5. Lessons learned and future work

Developing Rùa Toán Basic was an essential and transformative educational experience. Pivoting from the overly complex, framework-heavy architecture of the initial Next.js project to a plain HTML/CSS/JS and FastAPI foundation allowed the author to genuinely understand and appreciate the mechanics of web systems. Instead of relying on automated framework abstractions, the author learned how to construct resilient HTTP endpoints, design intuitive user interfaces using CSS custom properties and retro-modern aesthetics, safely handle binary file uploads with magic byte verification, and maintain stateless session cookies securely.

Key future extensions include:

- Integrating an automated GitHub Actions CI workflow to run `smoke_test.py` on every push.
- Connecting a transactional email service for automated password reset dispatch.
- Migrating binary photo storage to an S3-compatible blob store.
- Adding interactive practice quizzes with immediate feedback.

4.6. Conclusion

Rùa Toán Basic accomplishes its educational and architectural objectives with honesty and rigor. Rather than presenting a complex black-box application whose inner workings remain obscure, this project showcases an application that the author has built, debugged, and understood from the ground up. By combining lightweight technologies with exhaustive validation (26/26 automated smoke tests passed, 0 console errors in production, and zero compilation warnings), Rùa Toán Basic demonstrates that mastering foundational web technologies provides the strongest possible bedrock for future software engineering.

References

[1] FastAPI Contributors. Fastapi documentation, 2026. Accessed: 2026-11-01.

[2] MathJax Consortium. Mathjax: Beautiful and accessible math in all browsers, 2026. Accessed: 2026-11-01.

[3] Mozilla Developer Network. Mdn web docs: Html, css, and javascript, 2026. Accessed: 2026-11-01.

[4] PostgreSQL Global Development Group. Postgresql documentation: Data types, 2026. Accessed: 2026-11-01.

[5] Psycopg Contributors. Psycopg 3 documentation, 2026. Accessed: 2026-11-01.

[6] Uvicorn Contributors. Uvicorn: An asgi web server implementation for python, 2026. Accessed: 2026-11-01.

[7] Vercel. Vercel documentation, 2026. Accessed: 2026-11-01.

[8] Vercel. Vercel python runtime and fastapi, 2026. Accessed: 2026-11-01.

A. Appendix

A.1. Local setup and test execution

From a checkout of the Rùa Toán Basic repository:

```
# 1. create and activate python virtual environment
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

# 2. configure environment variables (.env.local)
cp .env.example .env.local
# edit .env.local: provide DATABASE_URL and SESSION_SECRET

# 3. run the automated smoke test suite (all 8 flows)
python scripts/smoke_test.py

# 4. run local development server
uvicorn main:app --reload --port 8000
# open http://127.0.0.1:8000 in browser
```

The database module reads `DATABASE_URL`; it creates schema `ruatoan_basic` and tables `users`, `photos`, and `password_resets` on first startup. The session requires `SESSION_SECRET`. Secrets are strictly isolated in ignored files and deployment environment variables.

A.2. Route inventory

Table 6 provides a compact route map for the pages and handlers implemented in Rùa Toán Basic.

A.3. Content inventory snapshot

The audited non-sample data files contained the following records on 1 November 2026:

- 11 source JSON files: 3,381 collections and 33,929 problem records;

Route	Purpose
/	Home: catalogue totals, mission, and marked problem bookmarks
/pages/practice.html	Problem search, grade/topic filters, pagination, and MathJax
/pages/exams.html	Collection list and detail view with ordered problems
/pages/lesson.html	Theory lessons with client search and expandable bodies
/pages/library.html	Reference books and documents with search
/pages/login.html, /pages/register.html	User authentication forms
/pages/submissions.html	Photo upload, thumbnail gallery, and deletion
/css/*, /js/*, /images/*	Static assets served directly by Vercel edge
/api/stats	Live catalogue totals and topic counts
/api/daily	Deterministic problem of the day
/api/collections, /api/collections/{id}	Collection list and detail with pagination
/api/problems	Problem search with folded diacritics and pagination
/api/lessons, /api/books, /api/docs	Reference catalogue collections
/api/register, /api/login, /api/logout, /api/me	Account session management
/api/change-password	Credential update with current password validation
/api/request-reset, /api/reset-password	Database-persisted single-use password reset
/api/photos, /api/photos/{id}	Photo upload (magic bytes), list, get, and delete

Table 6. Application page and API route map.

- 7 theory JSON files: 90 lessons;
- `books.json`: 216 books, magazines, and resource records;
- `docs.json`: 89 document records.

The topic assignment counts and collection breakdown are shown in Figures 11–13. These are repository measurements and should be regenerated if the content files change.

A.4. Data pipeline diagram

Figure 14 summarises how prepared content reaches the runtime pages.

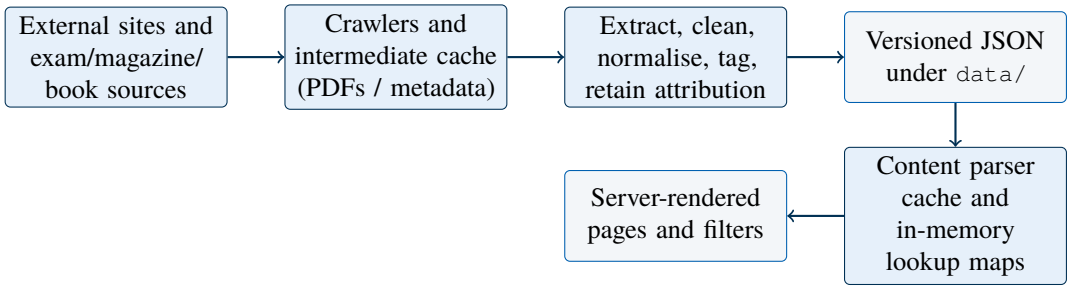


Figure 14. Content preparation and runtime consumption. The pipeline is an offline tool; it is not called during page requests. The basic version reuses its JSON output unchanged.

A.5. Glossary

Collection An ordered source-aware unit such as an exam paper or topic exercise set.

Problem One individually addressable statement nested in a collection.

Topic tag A classification label; one problem may have multiple tags.

Session cookie A signed `HttpOnly` cookie holding the user id and expiry.

Raw record count Count of JSON objects by record group before claims about uniqueness, correctness, or editorial review.

A.6. AI assistance and academic disclosure

This report draft was prepared with AI assistance (Claude) after inspecting the Rùa Toán Basic source and data files. The assistant helped organise and describe the implementation and performed the bug-fix and hardening pass described in Chapter 3; it did not run a full test suite, an accessibility audit, or an editorial review of the mathematics catalogue. The original project's content pipeline also contains exam-ingestion scripts that call Gemini and a Claude processing lane. The group should disclose any AI tools used during implementation, state which content or code they assisted with, and follow the course's academic-integrity rules.